

Java 반복문 / OOP / 클래스와 객체 / 메모리 구조

1. 반복문

반복문은 연속해서 반복되는 코드를 줄이기 위해 사용한다.

for문 기본 구조

```
for (변수 초기화; 조건식; 증감식) {  
    // 반복해서 수행할 코드  
}
```

예시

```
for (int i = 0; i < 5; i++) {  
    System.out.println("Hello");  
}
```

Hello를 5번 직접 작성하지 않고 반복문을 이용해서 처리할 수 있다.

여기저기 떨어져 있는 중복 코드는 어떻게 제거할까?

반복문은 주로 연속적으로 반복되는 코드를 처리한다.

하지만 프로그램의 여러 위치에서 똑같은 코드가 반복된다면 그 코드를 메소드(Method)로 만들어서 재사용할 수 있다.

예시

```
static void hello() {  
    System.out.println("안녕하세요");  
    System.out.println("반갑습니다");  
}
```

필요한 곳에서

```
hello();
```

처럼 호출한다.

즉,

연속적인 반복 → 반복문
여러 위치에서 반복되는 기능 → 메소드

2. Java 이름 작성 관례

Java에서는 클래스, 변수, 메소드의 이름을 작성할 때 일반적인 명명 규칙이 있다.

클래스명

첫 문자를 **대문자**로 작성한다.

```
Test  
Person  
StudentManager
```

여러 단어라면 각 단어의 첫 글자를 대문자로 작성한다.

```
StudentManager  
HelloJava
```

→ PascalCase

변수명 / 메소드명

첫 문자를 **소문자**로 작성한다.

```
age  
studentName  
printName()  
getScore()
```

여러 단어라면 두 번째 단어부터 첫 문자를 대문자로 작성한다.

```
studentName  
totalScore  
printMessage()
```

→ camelCase

3. Java를 왜 배우는가?

Java의 중요한 특징 중 하나는 객체 지향 프로그래밍(OOP)을 지원한다는 것이다.

OOP란?

Object Oriented Programming

→ 객체 지향 프로그래밍

프로그램을 여러 개의 객체(Object)로 구성하고,
객체들이 서로 데이터를 주고받거나 기능을 수행하도록 프로그램을 만드는 방식이다.

4. Class와 Object

클래스(Class)

객체를 만들기 위한 설계도라고 생각할 수 있다.

```
class Test {  
    int num;  
  
    void print() {  
        System.out.println(num);  
    }  
}
```

`Test` 클래스에는

데이터 → num
기능 → print()

이 정의되어 있다.

객체(Object)

클래스를 바탕으로 실제 메모리에 생성된 실체를 객체(Object)라고 한다.

```
Test t = new Test();
```

여기서

Test → 클래스
new Test() → Test 클래스를 이용해 새로운 객체 생성
t → 생성된 객체를 참조하는 변수

즉,

```
Test t = new Test();
```

를 간단하게 생각하면

Test라는 설계도를 이용해서
메모리에 실제 객체 하나를 만들어라.

라는 의미이다.

※ 입문 과정에서는

객체 = 메모리에 올라간 클래스 라고 이해해도 흐름은 비슷하지만,

더 정확하게는

클래스 → 객체를 만들기 위한 설계도
객체 → 클래스를 바탕으로 메모리에 생성된 실체

라고 이해하는 것이 좋다.

5. new

Java에서 new 는 객체를 생성할 때 사용하는 연산자이다.

```
new Test();
```

의 의미는

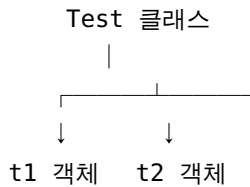
Test 클래스를 기반으로
새로운 Test 객체를 메모리에 생성한다.

이다.

예를 들어

```
Test t1 = new Test();  
Test t2 = new Test();
```

라고 하면 **Test** 클래스는 하나지만
서로 다른 객체가 2개 만들어진다.



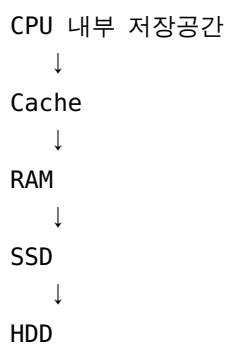
6. PC의 기본 구성 요소

컴퓨터의 주요 구성 요소를 간단하게 보면

CPU
Memory (RAM)
Main Board
SSD / HDD

등이 있다.

속도는 대략



순서로 CPU에 가까울수록 빠르다.

7. 프로그램은 어떻게 실행되는가?

프로그램 파일은 일반적으로 SSD나 HDD에 저장되어 있다.

예를 들어

```
SSD
└─ Test.class
```

와 같이 저장되어 있다고 하자.

CPU가 프로그램을 실행하기 위해서는
필요한 명령어와 데이터가 **메모리(RAM)**에 올라와 있어야 한다.

따라서 프로그램 실행 과정은 크게

```
SSD / HDD
  ↓
  RAM
  ↓
  CPU
```

와 같은 흐름을 가진다.

즉,

1. 프로그램이 SSD/HDD에 저장되어 있음
2. 프로그램을 실행함
3. 필요한 코드와 데이터가 RAM으로 올라옴
4. CPU가 RAM에 있는 명령어를 가져와 실행함

8. 왜 프로그램 전체를 RAM에 올리지 않을까?

프로그램의 크기가 매우 크다고 생각해보자.

예를 들어 프로그램 전체 크기가 100GB인데
현재 사용자가 실제로 사용하는 부분은 일부뿐일 수도 있다.

그런데 프로그램 전체를 한꺼번에 메모리에 올리면

```
필요하지 않은 코드까지 RAM 차지
  ↓
메모리 낭비
```

가 발생한다.

따라서 실제 컴퓨터에서는 **현재 필요한 코드와 데이터 중심으로 메모리에 적재하여 사용**한다.

운영체제의 가상 메모리, 페이지 등의 기능도 이러한 메모리 관리를 담당한다.

9. Java의 클래스와 메모리

Java 프로그램도 실행하려면 필요한 코드와 객체가 메모리에 존재해야 한다.

Java에서는 프로그램을 구성할 때 **클래스(Class)**를 기본적인 구조 단위로 사용한다.

```
class Test {  
    int num;  
  
    void print() {  
        System.out.println(num);  
    }  
}
```

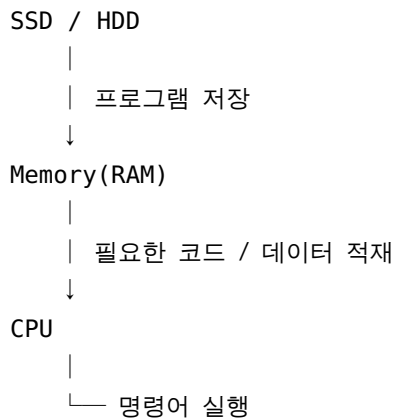
그리고 실제로 `Test` 객체가 필요하다면

```
Test t = new Test();
```

와 같이 작성한다.

이때 `new`를 통해 **Test** 클래스를 기반으로 객체를 생성하고 객체를 위한 메모리 공간을 확보한다.

10. 전체 흐름



Java 코드에서는

```
class Test{  
    ...  
}
```

클래스를 정의하고

```
Test t = new Test();
```

를 통해 실제 객체를 생성하여 사용한다.

핵심 정리

반복되는 연속 코드

→ 반복문(for, while)

여러 위치에서 반복되는 코드

→ 메소드로 분리하여 재사용

OOP

→ Object Oriented Programming

→ 객체 지향 프로그래밍

Class

→ 객체를 만들기 위한 설계도

Object

→ 클래스를 기반으로 메모리에 생성된 실체

new

→ 새로운 객체를 생성하는 연산자

프로그램 실행 흐름

SSD / HDD

↓

RAM

↓

CPU

특히 가장 중요하게 구분할 것:

```
Test t = new Test();
```

Test → 클래스 자료형
t → 객체를 가리키는 참조 변수
new → 객체 생성
new Test() → 생성된 Test 객체